

AZURE DATA + AI CAPSTONE

POC-08 Integrated Azure Data + AI Platform

Beginner implementation, Bicep deployment, verification, troubleshooting, and repeatable runbook

Poetry + Python + Azure Bicep + ADLS + ADF + Event Hubs + Databricks + Serving + Document Intelligence + Azure AI Search + Microsoft Foundry

How to use this guide

- **Follow the sequence.** Prove the Python pipeline locally first, then deploy Azure control-plane resources, then run data-plane tests.
- **Use the success condition on every step.** Do not continue after an error until the current step is green.
- **Run Python commands from the project root.** Running module commands from `infra\bicep` can cause import errors.
- **Use the beginner Bicep profile first.** SQL, Synapse, Databricks, ML, and Function App are optional/full-profile resources and can increase cost.
- **The final working chat model in this POC is Phi-4-mini-instruct.** The embedding deployment is text-embedding-3-small.

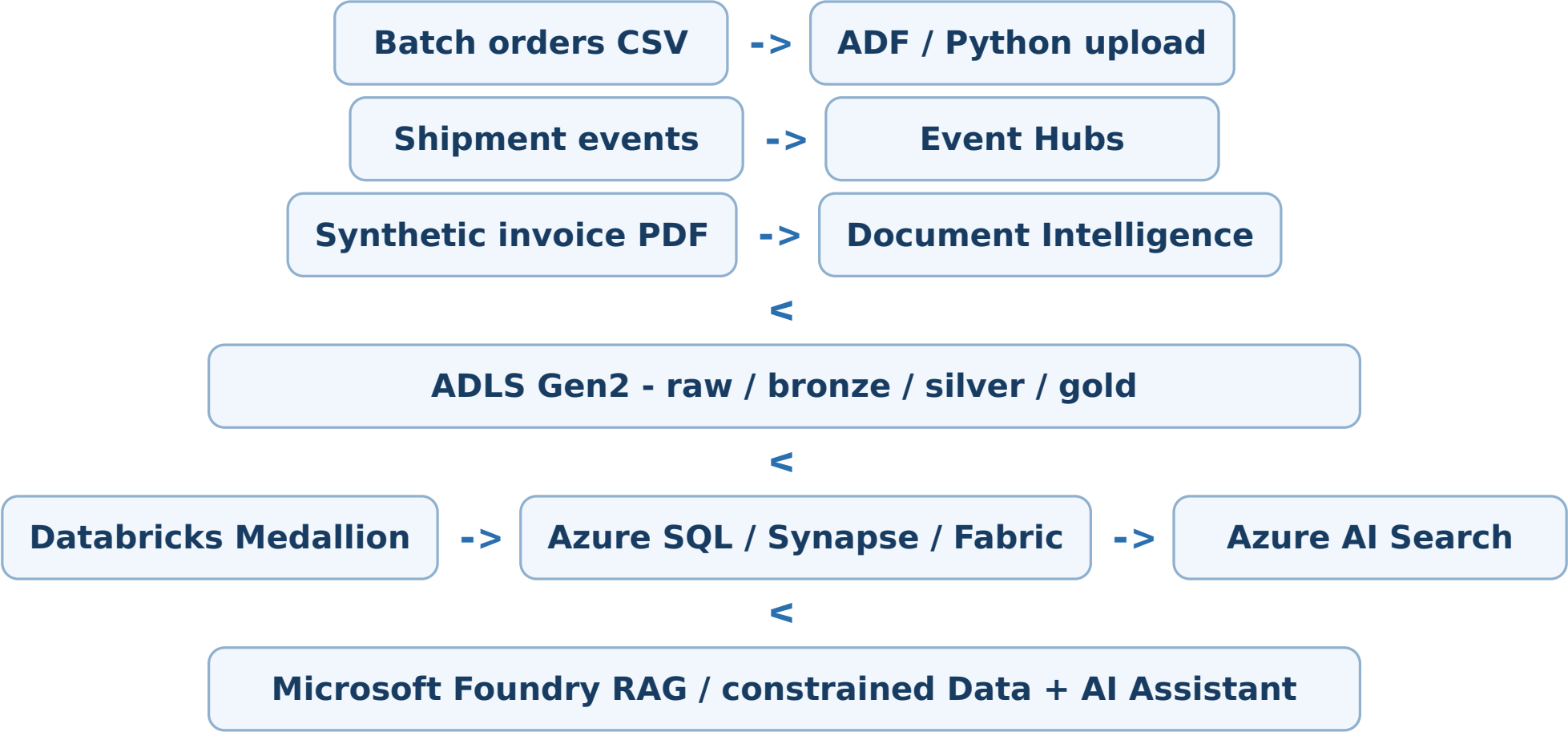
Table of contents

Section	What you will learn
1. Architecture	What POC-08 proves and how batch, stream, document, serving, Search, and Foundry connect.
2. Local setup	Poetry, .env, verify_config.py, synthetic data, tests, and smoke_test.py.
3. Azure Bicep	Subscription login, providers, build, validate, what-if, beginner/models/full profiles.
4. Data engineering	ADLS, ADF, Event Hubs, Databricks Bronze/Silver/Gold, SQL/Synapse/Fabric serving.
5. AI integration	Document Intelligence, embeddings, Azure AI Search, RAG, Foundry models, assistant.
6. Operations	RBAC, governance, monitoring, failure drill, CI/CD, cleanup.
7. Troubleshooting	Every important error hit during this POC and the working fix.
8. Command catalog	Runnable Python files, PowerShell scripts, Bicep modules, and final run order.

What the completed POC proves

- A synthetic orders CSV can move through a local and Azure batch path toward curated Gold metrics.
- Shipment events can be sent through Event Hubs and written to ADLS Bronze.
- A serving layer can expose curated Gold data through Azure SQL, Synapse, or Fabric.
- A synthetic invoice can be analyzed with Document Intelligence.
- Policy text and Gold metrics can be embedded and indexed in Azure AI Search.
- Hybrid/vector retrieval can return the correct grounding source.
- Microsoft Foundry can generate a grounded answer using the retrieved context.
- A constrained assistant can answer policy, metric, and mixed questions with read-only tools.
- The platform includes RBAC, governance, monitoring, CI/CD, failure recovery, and cost cleanup.

End-to-end architecture



Cross-cutting platform concerns

Concern	POC-08 implementation
Identity	Entra ID, interactive browser authentication for development, managed identities for workloads.
Access	RBAC matrix for Storage, Event Hubs, Search, Foundry, Document Intelligence, and SQL.
Secrets	Local .env for the lab; production guidance points to Key Vault/service connections/federated identity.
Governance	Dataset owner, classification, lineage, and access are documented.
Observability	Log Analytics, Application Insights, service metrics, and a local health check.
IaC	Azure Bicep is the primary control-plane deployment path.
CI/CD	Sanitized Azure DevOps YAML validates Python and Bicep and deploys only when explicitly enabled.
Cost	Beginner profile minimizes paid resources; full profile is opt-in; dedicated resource-group cleanup.

Project structure - core

```
azure-data-ai-capstone/  
|-- README.md  
|-- pyproject.toml  
|-- .env.example  
|-- architecture/  
|   |-- architecture.md  
|-- common/  
|   |-- auth.py  
|   |-- config.py  
|   |-- adls.py  
|   |-- embeddings.py  
|   |-- search_auth.py  
|-- data/  
|   |-- synthetic/  
|       |-- orders_001.csv  
|       |-- shipment_events.jsonl  
|       |-- policies.json  
|       |-- invoice_001.pdf
```

Project structure - ingestion and lakehouse

```
ingestion/  
|-- adf/  
|-- batch/  
|   `-- upload_orders.py  
|-- events/  
|   |-- schema.py  
|   |-- send_events.py  
|   |-- receive_events_to_adls.py  
|   `-- send_malformed_event.py  
`-- functions/  
    `-- function_app.py  
  
lakehouse/  
|-- local_medallion.py  
|-- bronze/01_bronze_orders.py  
|-- silver/02_silver_orders.py  
`-- gold/03_gold_orders.py
```


Project structure - serving and AI

```
serving/  
|-- sql/  
|   |-- schema.sql  
|   `-- load_gold.py  
|-- synapse/README.md  
`-- fabric/README.md  
  
ai/  
|-- document_intelligence/extract_invoice.py  
|-- rag/  
|   |-- prepare_documents.py  
|   |-- create_index.py  
|   |-- index_documents.py  
|   |-- query_search.py  
|   `-- rag_answer.py  
|-- agent/  
|   |-- assistant.py  
|   |-- foundry_agent_runner.py  
|   `-- tools.py  
`-- ml/delay_risk_tool.py
```

Project structure - infrastructure and operations

```
infra/bicep/  
|-- main.bicep  
|-- main.bicepparam  
|-- main.models.bicepparam  
|-- main.full.bicepparam  
|-- deploy_bicep.ps1  
|-- run_after_bicep.ps1  
|-- register_providers.ps1  
|-- configure_env_from_deployment.ps1  
|-- destroy_bicep.ps1  
`-- modules/*.bicep  
  
monitoring/  
security/  
governance/  
cicd/azure-pipelines.yml  
tests/  
scripts/
```

Prerequisites - Windows beginner setup

Verify the tools before creating any cloud resource.

- Python 3.12 is required by this project.
- Poetry manages the virtual environment and dependencies; pip-only setup is not required.
- Azure CLI is required for Bicep deployment and resource verification.
- Azure CLI Bicep support must be installed/upgraded before deployment.
- PowerShell is used for the provided Bicep helper scripts.
- Optional SQL path requires ODBC Driver 18 for SQL Server and the Poetry SQL dependency group.

Tool verification commands

```
py -3.12 --version  
poetry --version  
az --version  
az bicep version
```

```
# If Bicep is missing or old:  
az bicep install  
az bicep upgrade  
az bicep version
```

Poetry environment - create and install

Using "poetry run" avoids confusion about which interpreter is active.

```
# Run from the project root
poetry env use 3.12
poetry install
poetry env info
poetry run python --version

# Recommended: execute commands without manual activation
poetry run pytest

# Optional activation command
Invoke-Expression (poetry env activate)
```

Create the .env file

```
# PowerShell
Copy-Item .env.example .env

# CMD
copy .env.example .env

# Local validation does not require Azure values
poetry run python -m scripts.verify_config --profile local
```

Do not commit .env

The deployment helper can update the root .env from Bicep outputs and keys. Keep that file excluded from Git.

Local configuration check

```
(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone>poetry run python -m scripts.verify_config --profile local
[WARN] .env not found. Local-only commands can still run.

[LOCAL]
  OK - no Azure settings required

[SUCCESS] Configuration check passed for selected profile(s).

(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone>
```

Evidence: local profile contains no required Azure settings and the configuration check passes.

Generate synthetic inputs

```
poetry run python -m scripts.generate_synthetic_data  
Get-ChildItem data\synthetic
```

- `orders_001.csv` - batch retail orders.
- `shipment_events.jsonl` - three synthetic shipment events.
- `policies.json` - return and shipment-delay knowledge.
- `invoice_001.pdf` - synthetic invoice used by Document Intelligence.

Synthetic data generation evidence

```
(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone>poetry run python -m scripts.generate_synthetic_data  
Generated synthetic files in data\synthetic
```

The generator completes successfully and writes the sample files under data/synthetic.

Run code quality and unit tests

```
poetry run ruff check .  
poetry run pytest  
  
# Optional individual tests  
poetry run pytest tests/test_event_schema.py  
poetry run pytest tests/test_local_medallion.py  
poetry run pytest tests/test_agent_tools.py
```

Success condition

All tests pass before Azure deployment. This isolates cloud errors from Python logic errors.

Local Bronze -> Silver -> Gold smoke test

```
poetry run python -m scripts.smoke_test
Get-Content output\gold\gold_summary.json
```

Output	Purpose
output/bronze/orders.csv	Raw-normalized local Bronze proof.
output/silver/orders_clean.csv	Deduplicated and typed Silver data.
output/gold/customer_metrics.csv	Customer-level curated Gold metrics.
output/gold/gold_summary.json	Summary used by Search and the metric tool.

Local Medallion smoke-test evidence

```
(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone>poetry run python -m scripts.smoke_test
{
  "total_orders": 10,
  "total_revenue": 675.5,
  "average_order_value": 67.55,
  "generated_at": "2026-09-07T05:45:03.426878+00:00",
  "source": "output/gold/gold_summary.json"
}
[SUCCESS] Local batch -> Bronze -> Silver -> Gold smoke test passed.
```

Validated result: 10 orders, total revenue 675.50, average order value 67.55, and a successful local Bronze/Silver/Gold pipeline.

Initial Terraform trial - what happened

This section is retained because it was part of the real troubleshooting history.

- The first infrastructure attempt used Terraform and successfully initialized the AzureRM provider.
- Terraform validation also succeeded.
- The plan then prompted for `var.subscription_id` because the variable had no default and Terraform does not automatically read the Python `.env` file.
- The final POC direction changed to Azure Bicep so the active Azure CLI subscription could be used directly without a Terraform subscription variable prompt.

Earlier Terraform initialization

```
(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\terrafo
m>terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Finding hashicorp/random versions matching "~> 3.6"...
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by HashiCorp)
- Installing hashicorp/azurerm v4.81.0...
- Installed hashicorp/azurerm v4.81.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Historical evidence: Terraform initialized successfully before the IaC approach was changed to Bicep.

Earlier Terraform validation

```
(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\terraform>terraform validate
```

Warning: Argument is deprecated

```
with azurerm_key_vault.kv,  
on main.tf line 63, in resource "azurerm_key_vault" "kv":  
63:   enable_rbac_authorization = true
```

This property has been renamed to `rbac\_authorization\_enabled` and will be removed in v5.0 of the provider

Success! The configuration is valid, but there were some validation warnings as shown above.

Historical evidence: Terraform configuration validation succeeded.

Terraform subscription variable prompt

```
(azure-data-ai-capstone-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\terrafor
m>terraform plan -out poc08.tfplan
var.subscription_id
  Azure subscription id

Enter a value: █
```

Issue encountered: terraform plan asked for var.subscription_id. Final resolution for this POC: use the Bicep path and select the subscription with Azure CLI.

Azure login and subscription selection

```
az login
az account list -o table
az account set --subscription "<YOUR-SUBSCRIPTION-ID>"
az account show --query "{Subscription:name,SubscriptionId:id,TenantId:tenantId,User:user.name}" -o table
```

Bicep behavior

There is no Terraform-style `var.subscription_id` prompt. Subscription-scope Bicep uses the currently selected Azure CLI subscription.

Bicep deployment profiles

Profile	Creates	When to use
beginner	Resource Group, ADLS, Event Hubs, ADF, Search, Document Intelligence, Foundry resource/project, Key Vault, Log Analytics, Application Insights.	Run first. Lowest complexity and easiest troubleshooting.
models	Core resources plus Foundry chat and embedding deployments.	Run after core data/search checks and after confirming model availability/quota.
full	Adds Databricks, Azure SQL, Function App, Azure ML, and Synapse.	Optional. Can be materially more expensive.

Register Azure providers

```
cd infra\bicep
.\register_providers.ps1

# Optional explicit subscription
.\register_providers.ps1 -SubscriptionId "<YOUR-SUBSCRIPTION-ID>"

# Check remaining unregistered providers
az provider list --query "[?registrationState!='Registered'].[namespace,registrationState]" -o table
```

PowerShell signing issue - working fix

```
# Symptom:  
# register_providers.ps1 cannot be loaded because it is not digitally signed.  
  
# From project root, unblock the downloaded POC scripts:  
Get-ChildItem -Path . -Recurse -Filter *.ps1 | Unblock-File  
  
# Temporary policy change for this PowerShell process only:  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force  
  
cd infra\bicep  
.\register_providers.ps1
```

Why this is preferred

The process-scoped policy ends with the current PowerShell session; it does not permanently weaken the machine-wide policy.

Provider registration / Bicep readiness

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep
> .\register_providers.ps1
[OK] Microsoft.Resources already registered
[OK] Microsoft.Storage already registered
[OK] Microsoft.EventHub already registered
[OK] Microsoft.DataFactory already registered
[OK] Microsoft.Search already registered
[OK] Microsoft.CognitiveServices already registered
[OK] Microsoft.KeyVault already registered
[OK] Microsoft.OperationalInsights already registered
[OK] Microsoft.Insights already registered
[OK] Microsoft.Authorization already registered
[OK] Microsoft.Databricks already registered
[OK] Microsoft.Sql already registered
[OK] Microsoft.Web already registered
[OK] Microsoft.MachineLearningServices already registered
Registering Microsoft.Synapse ...
WARNING: Registering is still on-going. You can monitor using 'az provider show -n Microsoft.Synapse'

Provider registration was requested. Some providers can take a few minutes.
Check with: az provider list --query "[?registrationState!='Registered'].[namespace,registrationState]" -o table
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep
> █
```

Evidence captured after provider registration checks. Provider registration is a one-time subscription prerequisite for many Azure resource types.

Build, validate, and preview Bicep

```
az bicep build --file main.bicep

az deployment sub validate `
  --location eastus `
  --parameters main.bicepparam `
  --no-prompt true `
  --name poc08-validate

az deployment sub what-if `
  --location eastus `
  --parameters main.bicepparam `
  --no-prompt true `
  --name poc08-whatif
```

Important .bicepparam rule

The parameter file already contains a using declaration. Pass it through --parameters and do not also add --template-file main.bicep to the deployment command.

Deploy the beginner/core platform

```
# From infra\bicep  
.\deploy_bicep.ps1 -Profile beginner
```

- Checks the active Azure account.
- Registers providers.
- Builds Bicep.
- Runs subscription validation and what-if.
- Creates the Azure resources.
- Saves deployment-outputs.json and last-deployment.txt.
- Updates the root .env from Bicep outputs and service keys.

Azure resource group

The screenshot displays the Azure Resource Manager (ARM) console interface. On the left, the 'Resource Manager | Resource groups' sidebar is visible, showing a list of resource groups with 'rg-poc08-capstone' selected. The main pane shows the 'Overview' tab for this resource group. The top of the main pane includes a search bar, a 'Create' button, and a 'Manage view' dropdown. Below this, a list of resources is shown, but it is empty, displaying the message 'No resources match your filters'. The bottom of the main pane shows the status 'Showing 1 - 0 of 0. Display count: auto'.

Home > Resource Manager | Resource groups

Resource Manager | Resource groups

Default Directory

Search

+ Create Manage view

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

Showing 1 - 1 of 1. Display count: auto

Add or remove favorites by pressing Ctrl+Shift+F

rg-poc08-capstone

Resource group

Search

+ Create Manage view Delete resource group Refresh Group by none

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

JSON View

Resources Recommendations

Filter for any field...

Type equals all

Location equals all

Add filter

No resources match your filters

Try changing or clearing your filters.

Create Clear filters

Learn more

Showing 1 - 0 of 0. Display count: auto

Give feedback

Portal evidence: the dedicated POC resource group provides a single scope for verification and cleanup.

ADLS Gen2 storage and datalake container

The screenshot displays the Azure portal interface for the storage account **rgpoc08capstone**. The top navigation bar shows the account name and three helpful links: "Create an LCM rule for my storage account", "Help me reduce the costs of my storage account", and "Help me write an LCM rule for this storage account". Below the navigation bar, the left sidebar lists various management options, with "Overview" selected. The main content area is divided into two sections: "Essentials" and "Properties".

Essentials

Property	Value
Resource group (move)	: rg-poc08-capstone
Location	: eastus
Primary/Secondary Locat...	: Primary: East US, Secondary: West US
Subscription (move)	: Azure subscription 1
Subscription ID	: 9902e04b-fd65-40f4-86a2-1b09c1fe672c
Disk state	: Primary: Available, Secondary: Available
Tags (edit)	: Add tags

Properties

The "Properties" section is further divided into three sub-sections: "Data Lake Storage", "Security", and "Networking".

Data Lake Storage

Property	Value
Hierarchical namespace	Enabled
Default access tier	Hot
Blob anonymous access	Disabled
Blob soft delete	Enabled (7 days)
Container soft delete	Enabled (7 days)
Versioning	Disabled
Change feed	Disabled
NFS v3	Disabled

Security

Property	Value
Require secure transfer for REST API operations	Enabled
Storage account key access	Enabled
Minimum TLS version	Version 1.2
Infrastructure encryption	Disabled

Networking

Property	Value
Public network access	Enabled

Portal evidence: the storage account contains the datalake filesystem used for raw, bronze, silver, and gold paths.

Event Hubs shipment-events

Home

shipment-events | Overview

Deployment

Search

×

«

🗑️ Delete

🛑 Cancel

🔄 Redeploy

⬇️ Download

🔄 Refresh

Overview

Inputs

Outputs

Template

Deployment is in progress

Deployment name : shipment-events

Subscription : [Azure subscription 1](#)

Resource group : [rg-poc08-capstone](#)

Start time : 9/7/2026, 11:42:45 AM

Correlation ID : d7501126-e7d7-4909-b41f-6c842d3217e6

Deployment details

Resource	Type	Status	Operation details
 shipment-events	 Event Hubs Namespace	Created	Operation details



Microsoft Defender for Cloud

Secure your apps and infrastructure

[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials

[Start learning today >](#)

Work with an expert

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.

[Find an Azure expert >](#)

Portal evidence: the shipment-events Event Hub is deployed and ready for the streaming path.

Bicep resource verification

```
Active Azure account:
Subscription      SubscriptionId      TenantId      User
-----
Azure subscription 1 9902e04b-fd65-40f4-86a2-1b09c1fe672c 3e154330-72b5-42d6-8e2d-ad6be9ab706c er.irshad26@outlook.com
Developer object ID will be used for lab RBAC: 0cabca04-45aa-440e-9ce9-4b0e258604fb
[OK] Microsoft.Resources already registered
[OK] Microsoft.Storage already registered
[OK] Microsoft.EventHub already registered
[OK] Microsoft.DataFactory already registered
[OK] Microsoft.Search already registered
[OK] Microsoft.CognitiveServices already registered
[OK] Microsoft.KeyVault already registered
[OK] Microsoft.OperationalInsights already registered
[OK] Microsoft.Insights already registered
[OK] Microsoft.Authorization already registered
[OK] Microsoft.Databricks already registered
[OK] Microsoft.Sql already registered
[OK] Microsoft.Web already registered
[OK] Microsoft.MachineLearningServices already registered
[OK] Microsoft.Synapse already registered

Provider registration was requested. Some providers can take a few minutes.
Check with: az provider list --query "[?registrationState!='Registered'].[namespace,registrationState]" -o table

[1/4] Bicep version/build
Bicep CLI version 0.46.1 (545b338e2c)
```

CLI evidence: the core Azure resources are visible in the POC resource group after deployment.

Troubleshooting - Key Vault purge-protection failure

ERROR: The property "enablePurgeProtection" cannot be set to false.

```
# Original Bicep property
```

```
# enablePurgeProtection: false
```

```
# Working fix for this disposable lab:
```

```
# Remove the property entirely from modules/keyvault.bicep.
```

- Purge protection is irreversible after it is enabled.
- The lab should not explicitly set the property to false.
- After removing the property, rebuild, validate, what-if, and redeploy.

Key Vault deployment error evidence

```
eGroups/rg-poc08-capstone/providers/Microsoft.Resources/deployments/keyvault , message : At least one resource deployment operation failed. Please list deployment operations for details. Please see https://aka.ms/arm-deployment-operations for usage details.,"details":[{"code":"BadRequest","message":"The property \"enablePurgeProtection\" cannot be set to false. Enabling the purge protection for a vault is an irreversible action."}]}}]]}}}}
Bicep deployment create failed with exit code 1.
At C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep\deploy_bicep.ps1:17 char:9
+         throw "$Step failed with exit code $LASTEXITCODE."
+         ~~~~~
+ CategoryInfo          : OperationStopped: (Bicep deployment failed with exit code 1.:String) [], RuntimeException
+ FullyQualifiedErrorId : Bicep deployment create failed with exit code 1.
```

Sanitized evidence: the actionable failure was enablePurgeProtection being explicitly set to false.

Troubleshooting - Azure AI Search hostingMode warning

```
# Warning:  
# hostingMode expected 'Default' | 'HighDensity' but got 'default'  
  
# Fix in modules/search.bicep  
hostingMode: 'Default'
```

Impact

This warning was not the blocker, but correcting the value removed a Bicep type warning and kept the template aligned with the resource type definition.

Bicep deployment - provider/build stage

```
Resource changes: 15 to modify, 12 no change, 1 to ignore.
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep> .\deploy_bicep.ps1 -Profile beginner

Active Azure account:
Subscription      SubscriptionId      TenantId      User
-----
Azure subscription <subscription / tenant / user identifiers redacted>
Developer object ID
[OK] Microsoft.Resources already registered
[OK] Microsoft.Storage already registered
[OK] Microsoft.EventHub already registered
[OK] Microsoft.DataFactory already registered
[OK] Microsoft.Search already registered
[OK] Microsoft.CognitiveServices already registered
[OK] Microsoft.KeyVault already registered
[OK] Microsoft.OperationalInsights already registered
[OK] Microsoft.Insights already registered
[OK] Microsoft.Authorization already registered
[OK] Microsoft.Databricks already registered
[OK] Microsoft.Sql already registered
[OK] Microsoft.Web already registered
[OK] Microsoft.MachineLearningServices already registered
[OK] Microsoft.Synapse already registered

Provider registration was requested. Some providers can take a few minutes.
Check with: az provider list --query "[?registrationState!='Registered'].[namespace,registrationState]" -o table

[1/4] Bicep version/build
```

Evidence after fixes: provider checks complete and the deployment progresses into the Bicep build/deployment stages. Personal account identifiers are redacted.

Bicep deployment - resource changes

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep> az resource list ^
>> --resource-group rg-poc08-capstone ^
>> --output table
```

Name	ResourceGroup	Location	Type	Status
srch-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.Search/searchServices	Succeeded
docai-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.CognitiveServices/accounts	Succeeded
log-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.OperationalInsights/workspaces	Succeeded
evhns-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.EventHub/namespaces	Succeeded
ai-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.CognitiveServices/accounts	Succeeded
poc08capstone3da6gj	rg-poc08-capstone	eastus	Microsoft.Storage/storageAccounts	Succeeded
appi-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.Insights/components	Succeeded
adf-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.DataFactory/factories	Succeeded
ai-poc08capstone-3da6gj/poc08capstone-project	rg-poc08-capstone	eastus	Microsoft.CognitiveServices/accounts/projects	Succeeded
Application Insights Smart Detection	rg-poc08-capstone	global	microsoft.insights/actiongroups	Succeeded
kv-poc08capstone-3da6gj	rg-poc08-capstone	eastus	Microsoft.KeyVault/vaults	Succeeded

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep>
```

Evidence: resource creation/modification succeeds after correcting the Key Vault and Search module issues.

Bicep deployment - completed successfully

```
Name                               State    Timestamp                               Mode
-----
poc08-beginner-20260907-151429    Succeeded  2026-09-07T09:47:30.383317+00:00    Incremental

[SUCCESS] Bicep deployment completed: poc08-beginner-20260907-151429
Outputs saved to: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep\deployment-outputs.json
Deployment name saved to: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep\last-deployment.txt
[SUCCESS] Updated C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\.env from Bicep deployment outputs.
Secrets were written only to .env; .env must remain excluded from Git.

NEXT: from the project root run:
  poetry install
  poetry run python -m scripts.smoke_test
  .\infra\bicep\run_after_bicep.ps1 -DeploymentName poc08-beginner-20260907-151429
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep> 
```

Evidence: the beginner deployment succeeds, outputs are saved, last-deployment.txt is written, and the root .env is updated.

Verify core resources after Bicep

```
az group show -n rg-poc08-capstone -o table
az resource list -g rg-poc08-capstone -o table
az storage account list -g rg-poc08-capstone -o table
az eventhubs namespace list -g rg-poc08-capstone -o table
az datafactory list -g rg-poc08-capstone -o table
az search service list -g rg-poc08-capstone -o table
az cognitiveservices account list -g rg-poc08-capstone -o table
```

Run the post-Bicep data workflow

```
# From infra\bicep after a successful beginner deployment
cd ..\..\..

$deployment = (Get-Content .\infra\bicep\last-deployment.txt).Trim()

.\infra\bicep\run_after_bicep.ps1 `
    -DeploymentName $deployment
```

- Runs local tests/Gold generation.
- Uploads orders_001.csv to ADLS raw/orders.
- Runs the ADF raw -> bronze copy pipeline.
- Sends and receives Event Hubs shipment events.
- Runs Document Intelligence invoice extraction.
- Prepares Search documents and runs Search/RAG where models exist.
- Optionally loads Azure SQL and deploys Function code when enabled.

ADF CLI extension issue - root cause

```
# The first post-Bicep run printed:  
The command requires the extension datafactory. Do you want to install it now? (y/n)  
  
# The interactive prompt text was captured together with the ADF run ID.  
# Later pipeline-run show calls received an invalid run ID and returned Bad Request.
```

Important

The Bicep resources were already successful. This was a post-deployment Azure CLI/data-plane issue, not an infrastructure failure.

ADF post-Bicep failure evidence

```
OK - ADLS_FILE_SYSTEM
OK - ADLS_ACCOUNT_URL or connection string

[SUCCESS] Configuration check passed for selected profile(s).
[SUCCESS] Uploaded data\synthetic\orders_001.csv -> raw/orders/orders_001.csv
[4] Run ADF raw -> bronze copy pipeline
WARNING: Preview version of extension is disabled by default for extension installation, enabled for modules without stable versions.
WARNING: Please run 'az config set extension.dynamic_install_allow_preview=true or false' to config it specifically.

WARNING: Run 'az config set extension.use_dynamic_install=yes_without_prompt' to allow installing extensions without prompt.
ADF run id: The command requires the extension datafactory. Do you want to install it now? The command will continue to run after the extension is installed. (Y/n): a0a503ec-aa
a7-11f1-9cf4-580205bfcb2b
ERROR: Operation returned an invalid status 'Bad Request'
ADF status:
ERROR: Operation returned an invalid status 'Bad Request'
ADF status:
ERROR: Operation returned an invalid status 'Bad Request'
ADF status:
ERROR: Operation returned an invalid status 'Bad Request'
ADF status:
ERROR: Operation returned an invalid status 'Bad Request'
ADF status:

```

Evidence: the Azure CLI extension prompt contaminated the captured ADF run ID, followed by repeated Bad Request status checks.

ADF CLI extension issue - working fix

```
# Install the extension before capturing a pipeline run ID
az extension add --name datafactory --upgrade --yes

az extension show `
  --name datafactory `
  --query "{Name:name,Version:version}" `
  -o table

# Then rerun from the project root
$deployment = (Get-Content .\infra\bicep\last-deployment.txt).Trim()
.\infra\bicep\run_after_bicep.ps1 -DeploymentName $deployment
```

- The updated script checks/installs the datafactory extension first.
- It validates that the captured ADF run ID is a GUID before polling status.
- It stops with useful run details if the pipeline fails.

Azure Data Factory CLI extension verified

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> az extension add --name datafactory --upgrade --yes

Extension 'datafactory' 1.0.4 is already installed.
Latest version of 'datafactory' is already installed.
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> az extension show `
>> --name datafactory `
>> --query "{Name:name,Version:version}" `
>> -o table
Name      Version
-----
datafactory 1.0.4
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> █
```

Evidence: the datafactory Azure CLI extension is installed and available before rerunning the post-Bicep pipeline.

ADLS batch path - configure and verify

```
# .env
AZURE_AUTH_MODE=browser
ADLS_ACCOUNT_URL=https://<storage>.dfs.core.windows.net
ADLS_FILE_SYSTEM=datalake

# Verify configuration
poetry run python -m scripts.verify_config --profile adls

# Upload raw batch file
poetry run python -m ingestion.batch.upload_orders
```

Success condition

orders_001.csv exists under datalake/raw/orders in the storage account.

ADF raw -> bronze pipeline - manual verification

```
$deployment = (Get-Content .\infra\bicep\last-deployment.txt).Trim()
$outputs = az deployment sub show --name $deployment --query properties.outputs -o json |
ConvertFrom-Json
$rg = $outputs.resourceGroupName.value
$adf = $outputs.dataFactoryName.value
$pipeline = $outputs.dataFactoryPipelineName.value

$runId = (az datafactory pipeline create-run `
  -g $rg --factory-name $adf --name $pipeline `
  --query runId -o tsv --only-show-errors).Trim()

az datafactory pipeline-run show `
  -g $rg --factory-name $adf --run-id $runId `
  --query "{Status:status,RunId:runId,Pipeline:pipelineName}" -o table
```

ADF success condition

If the file reaches Bronze, the Azure batch ingestion path is proven.

- Pipeline status becomes **Succeeded**.
- `datalake/raw/orders/orders_001.csv` exists.
- `datalake/bronze/orders/orders_001.csv` exists.
- The ADF linked service uses managed identity and the deployment assigns the required storage role.

Event Hubs - configure

```
# .env
EVENTHUB_FULLY_QUALIFIED_NAMESPACE=<namespace>.servicebus.windows.net
EVENTHUB_NAME=shipment-events
EVENTHUB_CONSUMER_GROUP=$Default

poetry run python -m scripts.verify_config --profile events
```

- For Entra/browser authentication, the developer needs Data Sender and Data Receiver access.
- Connection-string fallback exists for a beginner lab, but identity-based access is preferred.

Event Hubs - send and receive

```
# Terminal 1 - receiver
poetry run python -m ingestion.events.receive_events_to_adls --max-events 3

# Terminal 2 - sender
poetry run python -m ingestion.events.send_events
```

Success condition

Three events are received and a JSONL file is written under datalake/bronze/events.

Streaming + document-processing success

```
[SUCCESS] Sent 3 shipment events to Event Hubs
Received E-001 from partition 0
Received E-002 from partition 0
Received E-003 from partition 0
[SUCCESS] Wrote 3 events to ADLS bronze/events
[6] Document Intelligence invoice extraction

[DOCUMENTS]
  OK - DOCUMENTINTELLIGENCE_ENDPOINT
  OK - DOCUMENTINTELLIGENCE_API_KEY

[SUCCESS] Configuration check passed for selected profile(s).
InvoiceId: INV-POC08-001
VendorName: Contoso Retail Pvt Ltd
InvoiceTotal: {'amount': 309.75, 'currencyCode': 'INR'}
[SUCCESS] Full extraction saved to output\document_intelligence\invoice_001_result.json
[7] Prepare RAG documents
[SUCCESS] Prepared 3 search documents -> output\search\documents.json
WARNING: Embedding deployment 'text-embedding-3-small' is not present. Use deploy_bicep.ps1 -Profile models after checking model availability/quota, then rerun this script.

[SUCCESS] Automated post-Bicep checks finished.
Databricks notebook import/run remains a data-plane handoff; follow docs\DATABRICKS_BICEP_HANDOFF.md.
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> █
```

Evidence: three shipment events were sent/received and written to ADLS Bronze; Document Intelligence extraction also succeeded. The remaining warning at this stage was only the missing embedding model deployment.

Databricks Medallion - notebook order

1. Import `lakehouse/bronze/01_bronze_orders.py`.
2. Import `lakehouse/silver/02_silver_orders.py`.
3. Import `lakehouse/gold/03_gold_orders.py`.
4. Attach temporary compute.
5. Grant the workspace identity/access method used in the earlier Databricks POC to ADLS.
6. Set the notebook widget paths to the POC storage account.
7. Run Bronze -> Silver -> Gold in order.
8. Terminate compute when the validation is complete.

Databricks Medallion - ABFSS paths

```
abfss://datalake@<storage>.dfs.core.windows.net/raw/orders
abfss://datalake@<storage>.dfs.core.windows.net/bronze/orders
abfss://datalake@<storage>.dfs.core.windows.net/silver/orders
abfss://datalake@<storage>.dfs.core.windows.net/gold/customer_metrics
```

Layer	Verify
Bronze	Row count > 0; source_file and ingestion_ts are present.
Silver	Duplicates removed; types cast; invalid quantity/price filtered; line_amount created.
Gold	One row per customer with total_orders, total_units, and total_revenue.

Serving layer - choose one minimal path

Option	Best use in this POC	Automation
Azure SQL	Simple relational serving and read-only analytical tool.	Optional Bicep full profile + Python loader.
Synapse	Serverless SQL view/external table over curated lake data.	Optional Bicep full profile; can reuse prior workspace.
Fabric	OneLake/Lakehouse/Warehouse serving and SQL endpoint.	Reuse/create Fabric workspace; data-plane setup is separate from this Bicep lab.

Minimal capstone rule
One serving layer is sufficient for the demo. You do not need to pay for all three simultaneously.

Azure SQL - create with the full profile

```
# From PowerShell before the full profile
$env:POC08_SQL_ADMIN_PASSWORD="<STRONG-TEMPORARY-LAB-PASSWORD>"
$env:POC08_CLIENT_IP="<YOUR-PUBLIC-IP>" # optional

cd infra\bicep
.\deploy_bicep.ps1 -Profile full
```

- The SQL Bicep module creates a logical SQL server and database.
- The default database SKU is Basic unless changed.
- An AllowAzureServices firewall rule is created; an optional local developer IP rule can also be created.
- The full profile also enables other optional resources, so review what-if and cost before applying.

Azure SQL - load and query Gold

```
cd ..\..\npoetry install --with sql\n\n# .env example\nAZURE_SQL_ODBC_CONNECTION_STRING=Driver={ODBC Driver 18 for SQL\nServer};Server=tcp:<server>.database.windows.net,1433;Database=<db>;Encrypt=yes;TrustServerCertifica\nte=no;Authentication=ActiveDirectoryInteractive;\n\npoetry run python -m serving.sql.load_gold
```

```
SELECT *\nFROM dbo.v_customer_metrics\nORDER BY total_revenue DESC;
```

Success condition

The curated customer metrics are visible through the SQL view and match the Gold output.

Synapse serving option

1. Reuse a Synapse workspace from an earlier POC when possible to reduce cost.
2. If creating through the full Bicep profile, the workspace uses the POC ADLS account as default data lake storage.
3. Grant the Synapse managed identity Storage Blob Data Contributor access to the storage account.
4. Expose the Gold files or Delta location through a serverless SQL external table/view.
5. Run one curated query such as total revenue by customer and capture the result.

Success condition

A serverless analytical query returns the current Gold data.

Fabric / OneLake serving option

1. Reuse an existing Fabric workspace when available.
2. Create or reuse a Lakehouse/Warehouse inside the workspace.
3. Mirror or create a shortcut to the curated Gold output in OneLake.
4. Expose the data through the Lakehouse/Warehouse SQL endpoint.
5. Run one curated query and capture the result for the capstone demo.

Important

The runnable Bicep implementation automates Azure SQL/Synapse, not Fabric workspace data-plane provisioning. Fabric is a valid alternative serving path.

Document Intelligence - configure and run

```
# .env
DOCUMENTINTELLIGENCE_ENDPOINT=https://<name>.cognitiveservices.azure.com/
DOCUMENTINTELLIGENCE_API_KEY=<KEY>

poetry run python -m scripts.verify_config --profile documents

poetry run python -m ai.document_intelligence.extract_invoice `
  --file data/synthetic/invoice_001.pdf`
```

Success condition

output/document_intelligence/invoice_001_result.json exists and invoice ID, vendor, and total are extracted reasonably.

Prepare RAG documents

```
poetry run python -m scripts.smoke_test  
poetry run python -m ai.rag.prepare_documents  
Get-Content output\search\documents.json
```

- Policy documents become searchable knowledge.
- The latest Gold summary becomes an analytical knowledge item.
- The resulting documents are later embedded and loaded into Azure AI Search.

Search and embedding configuration

```
# .env
AZURE_SEARCH_ENDPOINT=https://<search>.search.windows.net
AZURE_SEARCH_INDEX=capstone-knowledge
AZURE_SEARCH_ADMIN_KEY=<ADMIN-KEY-OR-BLANK-FOR-RBAC>
AZURE_SEARCH_QUERY_KEY=<QUERY-KEY-OR-BLANK-FOR-RBAC>

AZURE_OPENAI_ENDPOINT=https://<ai-resource>.openai.azure.com
AZURE_OPENAI_API_KEY=<KEY-OR-RBAC>
AZURE_OPENAI_EMBEDDING_DEPLOYMENT=text-embedding-3-small
SEARCH_VECTOR_DIMENSIONS=1536
```

Create, load, and query the Search index

```
poetry run python -m scripts.verify_config --profile search
poetry run python -m scripts.verify_config --profile embeddings
```

```
poetry run python -m ai.rag.create_index
poetry run python -m ai.rag.index_documents
```

```
poetry run python -m ai.rag.query_search `
    "When is a shipment considered delayed?"
```

```
poetry run python -m ai.rag.query_search `
    "What is the current total revenue?"
```

Success condition

The shipment query returns policy-shipping.md and the metric query returns the Gold summary source.

Foundry model deployment - parent operation failure

```
# Symptom during .\deploy_bicep.ps1 -Profile models  
Another operation is being performed on the parent resource.  
Please try again later.
```

- The Foundry project, chat deployment, and embedding deployment are child writes under the same AI Services parent resource.
- The original Bicep allowed child writes to overlap.
- Azure rejected concurrent writes against the same parent.

Foundry parent-operation error evidence

```
Bicep deployment create failed with exit code 1.  
At  
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep\deploy_bicep.ps1:17  
char:9  
+         throw "$Step failed with exit code $LASTEXITCODE."  
+         ~~~~~  
+ CategoryInfo          : OperationStopped: (Bicep deploymen...th exit code 1.:String) [], RuntimeException  
+ FullyQualifiedErrorId : Bicep deployment create failed with exit code 1.
```

Sanitized terminal evidence from the failed model profile deployment. The actionable problem was concurrent operations on the AI Services parent resource.

Foundry model deployment - working serialization fix

```
# In modules/foundry.bicep
# Chat deployment waits for the Foundry project

dependsOn: [
  project
]

# Embedding deployment waits for the chat deployment

dependsOn: [
  chatDeployment
]
```

Result

The resource sequence becomes: AI Services account -> Foundry project -> chat deployment -> embedding deployment.

Final model profile for this POC

```
# Chat model
POC08_CHAT_DEPLOYMENT=Phi-4-mini-instruct
POC08_CHAT_MODEL=Phi-4-mini-instruct
POC08_CHAT_MODEL_VERSION=1
POC08_CHAT_MODEL_FORMAT=Microsoft
POC08_CHAT_SKU=GlobalStandard

# Embedding model
POC08_EMBEDDING_DEPLOYMENT=text-embedding-3-small
POC08_EMBEDDING_MODEL=text-embedding-3-small
POC08_EMBEDDING_MODEL_VERSION=1
POC08_EMBEDDING_MODEL_FORMAT=OpenAI
POC08_EMBEDDING_SKU=Standard
```

Deploy the final Foundry models

```
cd infra\bicep

$env:POC08_CHAT_DEPLOYMENT="Phi-4-mini-instruct"
$env:POC08_CHAT_MODEL="Phi-4-mini-instruct"
$env:POC08_CHAT_MODEL_VERSION="1"
$env:POC08_CHAT_MODEL_FORMAT="Microsoft"
$env:POC08_CHAT_SKU="GlobalStandard"

.\deploy_bicep.ps1 -Profile models
```

Before retrying

If a prior model deployment failed, first confirm the Foundry parent provisioningState is Succeeded. Model/version/SKU availability can vary by subscription and region.

Foundry resource verification

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep> az cognitiveservices account show `
>> --resource-group rg-poc08-capstone `
>> --name ai-poc08capstone-3da6gj `
>> --query "{Name:name,State:properties.provisioningState,Location:location,Kind:kind}" `
>> -o table
Name                               State      Location    Kind
-----
ai-poc08capstone-3da6gj  Succeeded  eastus      AIServices
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone\infra\bicep> 
```

Evidence: the POC-08 Foundry/AI Services resource exists and is in a successful state.

Both AI model deployments verified

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> az cognitiveservices account deployment show `
>> --resource-group $rg `
>> --name $foundry `
>> --deployment-name "Phi-4-mini-instruct" `
>> --query "{Deployment:name,State:properties.provisioningState,Model:properties.model.name,Format:properties.model.format,Version:properties.model.version}" `
>> -o table
Deployment          State      Model          Format      Version
-----
Phi-4-mini-instruct Succeeded Phi-4-mini-instruct Microsoft 1
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> az cognitiveservices account deployment list `
>> --resource-group $rg `
>> --name $foundry `
>> --query "[].{Deployment:name,State:properties.provisioningState,Model:properties.model.name,Format:properties.model.format,Version:properties.model.version}" `
>> -o table
Deployment          State      Model          Format      Version
-----
text-embedding-3-small Succeeded text-embedding-3-small OpenAI 1
Phi-4-mini-instruct Succeeded Phi-4-mini-instruct Microsoft 1
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> 
```

Evidence: Phi-4-mini-instruct and text-embedding-3-small are both present in the POC-08 AI Services resource.

Why gpt-5-mini was skipped

- The earlier project default looked for a chat deployment named `gpt-5-mini`.
- The embedding deployment existed, so Search indexing and hybrid/vector retrieval were already successful.
- RAG answer generation was skipped only because the expected chat deployment name was missing.
- The final fix aligned the POC with the previously working deployment: `Phi-4-mini-instruct`.
- The code also accepts `FOUNDRY_CHAT_DEPLOYMENT` and keeps `FOUNDRY_MODEL_DEPLOYMENT` as a compatible alias.

Missing gpt-5-mini warning before final fix

```
OK - AZURE_OPENAI_ENDPOINT
OK - AZURE_OPENAI_EMBEDDING_DEPLOYMENT

[SUCCESS] Configuration check passed for selected profile(s).
[SUCCESS] Search index ready: capstone-knowledge
[SUCCESS] Indexed 3 documents

#1 Shipment Delay Policy [policy-shipping.md]
A shipment is considered delayed when it misses the promised delivery date by more than 24 hours. Customer support should proactively notify the customer for delays above 48 hours.

#2 Retail Return Policy [policy-return.md]
Unused items may be returned within 30 days of delivery with proof of purchase. Opened electronics are accepted within 14 days unless defective.

#3 Current Gold Order Metrics [output/gold/gold_summary.json]
The curated Gold dataset contains 10 orders, total revenue 675.5, and average order value 67.55. Generated at 2026-09-07T12:00:56.025440+00:00.

[SUCCESS] Hybrid/vector retrieval returned grounded sources.
WARNING: Chat deployment 'gpt-5-mini' is not present, so RAG answer generation is skipped.

[SUCCESS] Automated post-Bicep checks finished.
Databricks notebook import/run remains a data-plane handoff; follow docs\DATABRICKS_BICEP_HANDOFF.md.
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone>
```

Evidence: Search and retrieval passed, but answer generation was skipped because gpt-5-mini was not present.

Final .env model settings

```
FOUNDRY_PROJECT_ENDPOINT=https://<foundry-resource>.services.ai.azure.com/api/projects/<project-name>
FOUNDRY_CHAT_DEPLOYMENT=Phi-4-mini-instruct
FOUNDRY_MODEL_DEPLOYMENT=Phi-4-mini-instruct

AZURE_OPENAI_ENDPOINT=https://<ai-resource>.openai.azure.com
AZURE_OPENAI_EMBEDDING_DEPLOYMENT=text-embedding-3-small
```

Verify the chat name Python reads

```
poetry run python -c "from common.config import Settings; print(Settings().foundry_model)"
```

Expected:

Phi-4-mini-instruct

Run the grounded RAG answer

```
poetry run python -m ai.rag.rag_answer `
    "When is a shipment considered delayed?"

# Unsupported grounding test
poetry run python -m ai.rag.rag_answer `
    "What is the employee vacation policy?"
```

- Supported question: answer should use the retrieved shipment policy and cite its source.
- Unsupported question: answer should state that the supplied context is insufficient instead of inventing a policy.

Grounded RAG answer success

```
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> poetry run python -c "from common.config import Settings; print(Settings().foundry_model)"
Phi-4-mini-instruct
(azure-data-ai-capstone-py3.12) PS C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\08_azure-data-ai-capstone> poetry run python -m ai.rag.rag_answer `
>> "When is a shipment considered delayed?"
A shipment is considered delayed when it misses the promised delivery date by more than 24 hours [policy-shipping.md].

Retrieved sources:
- policy-shipping.md
- policy-return.md
- output/gold/gold_summary.json
```

Final evidence: retrieval returns the shipment policy and the Phi chat model generates a grounded answer with the expected source list.

Constrained Data + AI Assistant

Policy question

```
poetry run python -m ai.agent.assistant --mode policy `
    "What is the return window for opened electronics?"
```

Metric question

```
poetry run python -m ai.agent.assistant --mode metric `
    "What is total revenue?"
```

Mixed question

```
poetry run python -m ai.agent.assistant --mode mixed `
    "What is total revenue and when should support notify a customer about a shipment delay?"
```

Assistant safety design

- Knowledge tool: read-only Azure AI Search retrieval.
- Metric tool: read-only access to the curated Gold summary.
- The assistant does not expose create/update/delete tools.
- Unsupported questions should remain unsupported rather than producing fabricated business rules.
- Optional POC-07 delay-risk scoring is also exposed only as a read-only scoring interface.

Optional Foundry Agent Service runner

```
# Only after an agent has been created in Foundry and configured  
FOUNDRY_AGENT_NAME=<agent-name>
```

```
poetry run python -m ai.agent.foundry_agent_runner `  
    "What is the return policy?"
```

Optional

The application-owned constrained assistant is sufficient for the core capstone. The Foundry Agent Service runner is an optional extension.

Optional POC-07 delay-risk scoring

```
# Offline wiring proof
poetry run python -m ai.ml.delay_risk_tool

# Real MLflow integration
poetry install --with ml

# .env
MLFLOW_MODEL_URI=<your-poc07-model-uri>
```

- The interface is read-only and returns a risk score.
- If no MLflow model URI is set, the POC uses a deterministic offline demo heuristic only for wiring validation.
- Do not expose model training or update operations to the assistant.

Governance record

Dataset	Owner	Classification / lineage
Orders raw	Data Engineering	Internal synthetic; source -> upload/ADF -> ADLS raw.
Orders Silver/Gold	Data Engineering / Analytics	Raw -> Databricks Bronze -> Silver -> Gold -> serving/search.
Shipment events	Data Engineering	Event Hubs -> ADLS Bronze.
Invoice extraction	AI Engineering	PDF -> Document Intelligence -> JSON/search.
Policy knowledge	Business/Data Owner	JSON -> embeddings -> Search -> RAG.

RBAC checklist

Workload	Minimum demo access
Developer / ADLS	Storage Blob Data Contributor.
Event sender	Azure Event Hubs Data Sender.
Event receiver	Azure Event Hubs Data Receiver.
Search index builder	Search Service Contributor + Search Index Data Contributor.
Search runtime	Search Index Data Reader.
Foundry app/user	Azure AI User or approved equivalent.
SQL runtime	Contained read-only user with SELECT only.

Threat model - practical controls

- **Secret leakage:** .env is gitignored; production secrets move to Key Vault/service connections/federated identity.
- **Over-privilege:** separate sender, receiver, reader, and contributor roles.
- **Prompt injection / unsupported questions:** answer only from retrieved context and explicitly report unsupported requests.
- **Malformed data:** validate event schema before Bronze write.
- **Accidental AI writes:** expose only retrieval, metric, and scoring functions.
- **Cost runaway:** keep paid compute optional and clean up model/search/compute resources after the lab.

Monitoring and health checks

```
poetry run python -m monitoring.health_check
```

- Latest batch status and row count.
- Newest shipment event timestamp / stream freshness.
- Document Intelligence extraction success.
- Search query latency and correct source retrieval.
- Foundry/model latency or trace.
- Tool failure count / expected malformed-event failure.
- Azure Cost Management review before cleanup.

Controlled failure drill

```
# Safest failure drill  
poetry run python -m ingestion.events.send_malformed_event  
  
# Recovery proof  
poetry run python -m ingestion.events.send_events
```

- Expected malformed event is rejected because required fields are missing.
- Recovery is demonstrated by sending valid events successfully afterward.
- Alternative drill: ask an unsupported RAG question and verify that the assistant refuses to invent context.

CI/CD - Azure DevOps pipeline

- The repository contains `cicd/azure-pipelines.yml`.
- It validates the Poetry/Python project, Ruff, unit tests, and Bicep compilation.
- Deployment is blocked unless `ENABLE_DEPLOY=true`.
- Credentials belong in an Azure DevOps service connection, variable group, or federated identity - not in YAML.
- Replace the placeholder service-connection name before using the deployment stage.

Python command catalog - local and ingestion

File	How to run / purpose
scripts/verify_config.py	<code>poetry run python -m scripts.verify_config --profile local adls events documents search embeddings foundry</code>
scripts/generate_synthetic_data.py	<code>poetry run python -m scripts.generate_synthetic_data</code> - regenerate CSV/JSON inputs.
scripts/smoke_test.py	<code>poetry run python -m scripts.smoke_test</code> - local Bronze/Silver/Gold proof.
ingestion/batch/upload_orders.py	<code>poetry run python -m ingestion.batch.upload_orders</code> - upload orders to ADLS raw.
ingestion/events/send_events.py	<code>poetry run python -m ingestion.events.send_events</code> - send three shipment events.
ingestion/events/receive_events_to_adls.py	<code>poetry run python -m ingestion.events.receive_events_to_adls --max-events 3</code> - write Event Hubs messages to Bronze.
ingestion/events/send_malformed_event.py	<code>poetry run python -m ingestion.events.send_malformed_event</code> - controlled failure drill.

Python command catalog - AI and serving

File	How to run / purpose
ai/document_intelligence/extract_invoice.py	<code>poetry run python -m ai.document_intelligence.extract_invoice --file data/synthetic/invoice_001.pdf</code>
ai/rag/prepare_documents.py	<code>poetry run python -m ai.rag.prepare_documents</code> - build searchable docs.
ai/rag/create_index.py	<code>poetry run python -m ai.rag.create_index</code> - create/update vector index.
ai/rag/index_documents.py	<code>poetry run python -m ai.rag.index_documents</code> - embed and upload documents.
ai/rag/query_search.py	<code>poetry run python -m ai.rag.query_search "question"</code> - hybrid/vector retrieval.
ai/rag/rag_answer.py	<code>poetry run python -m ai.rag.rag_answer "question"</code> - grounded Phi answer.
serving/sql/load_gold.py	<code>poetry install --with sql</code> , then <code>poetry run python -m serving.sql.load_gold</code> .

Python command catalog - assistant, ML, monitoring

File	How to run / purpose
ai/agent/assistant.py	<code>poetry run python -m ai.agent.assistant --mode policy metric mixed "question".</code>
ai/agent/foundry_agent_runner.py	<code>poetry run python -m ai.agent.foundry_agent_runner "question"</code> after FOUNDRY_AGENT_NAME is set.
ai/ml/delay_risk_tool.py	<code>poetry run python -m ai.ml.delay_risk_tool</code> - optional read-only risk scoring.
monitoring/health_check.py	<code>poetry run python -m monitoring.health_check</code> - local artifact freshness/status.
tests/test_event_schema.py	<code>poetry run pytest tests/test_event_schema.py.</code>
tests/test_local_medallion.py	<code>poetry run pytest tests/test_local_medallion.py.</code>
tests/test_agent_tools.py	<code>poetry run pytest tests/test_agent_tools.py.</code>

Python files that are libraries or platform entry points

File	Run guidance
common/auth.py, config.py, adls.py, embeddings.py, search_auth.py	Imported by runnable modules; do not normally execute directly.
ingestion/events/schema.py	Shared event validation model; exercised by sender/receiver/tests.
ai/agent/tools.py	Read-only metric helper imported by assistant.py.
lakehouse/local_medallion.py	Transformation library called by smoke_test.py.
ingestion/functions/function_app.py	Azure Functions application entry point; deploy with the Function deployment script rather than running as a normal CLI module.
__init__.py files	Package markers; no direct execution needed.

Databricks Python notebooks - run in workspace

Notebook	Purpose / verification
lakehouse/bronze/ 01_bronze_orders.py	Read raw orders, add ingestion metadata, write Delta Bronze; verify row count and metadata columns.
lakehouse/silver/ 02_silver_orders.py	Deduplicate, cast types, filter invalid rows, create line_amount; verify clean Silver rows.
lakehouse/gold/03_gold_orders.py	Aggregate customer metrics; verify total_orders, total_units, total_revenue by customer.

PowerShell / Bicep command catalog

Script	Purpose
infra/bicep/register_providers.ps1	Register required Azure resource providers.
infra/bicep/deploy_bicep.ps1	Deploy beginner, models, or full Bicep profile.
infra/bicep/configure_env_from_deployment.ps1	Populate/update root .env from a successful deployment.
infra/bicep/run_after_bicep.ps1	Run the post-infrastructure data-plane workflow.
infra/bicep/deploy_function_code.ps1	Zip-deploy optional Azure Function code.
infra/bicep/deploy_and_run_beginner.ps1	One-click beginner infrastructure + post-deployment data flow.
infra/bicep/destroy_bicep.ps1	Delete the dedicated POC resource group after explicit confirmation.

Bicep module catalog - core

Module	Resource responsibility
storage.bicep	ADLS Gen2 storage account + datalake filesystem.
eventhub.bicep	Event Hubs namespace and shipment-events hub.
datafactory.bicep	ADF, managed-identity ADLS linked service, datasets, raw-to-bronze pipeline.
search.bicep	Azure AI Search service.
document-intelligence.bicep	Document Intelligence account.
foundry.bicep	AI Services/Foundry resource, project, optional chat + embedding deployments.
keyvault.bicep	Key Vault for production-style secret management pattern.
monitoring.bicep	Log Analytics + Application Insights.

Bicep module catalog - optional full lab

Module	Resource responsibility
databricks.bicep	Optional Azure Databricks workspace.
sql.bicep	Optional Azure SQL logical server, database, firewall rules.
synapse.bicep	Optional Synapse workspace bound to the POC storage account.
functionapp.bicep	Optional Linux Function App infrastructure.
machine-learning.bicep	Optional Azure Machine Learning workspace.

Troubleshooting matrix - environment and IaC

Symptom	Cause	Working fix
terraform plan asks var.subscription_id	Terraform variable had no default; .env is not automatically a Terraform variable source.	Use tfvars if staying on Terraform; final POC uses Bicep + az account set.
ModuleNotFoundError: No module named scripts	Python module command was run from infra/bicep instead of project root.	cd ..\..\, then rerun poetry run python -m scripts.smoke_test.
PowerShell script is not digitally signed	Windows execution policy / downloaded-file block.	Unblock-File and use process-scoped Bypass for the current session.
Provider not registered	Subscription resource provider not registered yet.	Run register_providers.ps1 and wait until required providers show Registered.

Troubleshooting matrix - Bicep and ADF

Symptom	Cause	Working fix
enablePurgeProtection cannot be set to false	Key Vault API rejects explicit false.	Remove the property from keyvault.bicep for the disposable lab.
Search hostingMode warning	Value was lowercase default.	Use hostingMode: Default.
ADF run ID contains extension prompt	datafactory CLI extension was installed interactively while output was being captured.	Install/upgrade extension first; validate captured run ID is a GUID.
ADF pipeline-run show returns Bad Request	Invalid contaminated run ID was passed to the status command.	Fix extension/run-ID handling, launch a fresh ADF run, then poll status.

Troubleshooting matrix - Foundry and paid SKUs

Symptom	Cause	Working fix
Another operation is being performed on parent resource	Concurrent Foundry child writes under one AI Services account.	Serialize project -> chat -> embedding deployment using dependsOn.
Chat deployment gpt-5-mini is not present	Project expected a deployment that did not exist in this resource.	Use Phi-4-mini-instruct and align .env/code/Bicep deployment name.
Search free SKU creation fails	Subscription may already have free-tier limitation.	Use Basic only if necessary and clean it up after the POC.
Document Intelligence F0 creation fails	Subscription may already have the permitted free account.	Use S0 only if needed; remember it is billable.
Model profile fails with quota/version/SKU error	Model availability and quota vary by region/subscription.	Check the current resource/region; deploy an available model/version/SKU rather than changing unrelated resources.

Non-blocking Bicep warnings seen during the POC

These warnings were not the fatal model-deployment error. Prioritize the actual ERROR / failed provisioning state first.

- BCP187 on a resource-type property definition.
- no-hardcoded-env-urls linter warning in SQL output formatting.
- use-resource-symbol-reference linter warning in the Function module.
- no-unnecessary-dependson linter warning.
- BCP037 on a Data Factory linked-service type definition.

Clean-run command sequence - local proof

```
# Project root
poetry env use 3.12
poetry install
Copy-Item .env.example .env -ErrorAction SilentlyContinue

poetry run python --version
poetry run ruff check .
poetry run pytest
poetry run python -m scripts.generate_synthetic_data
poetry run python -m scripts.verify_config --profile local
poetry run python -m scripts.smoke_test
```

Clean-run command sequence - deploy core Bicep

```
az login
az account list -o table
az account set --subscription "<YOUR-SUBSCRIPTION-ID>"
az account show -o table

Get-ChildItem -Path . -Recurse -Filter *.ps1 | Unblock-File
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force

cd infra\bicep
.\register_providers.ps1
az bicep build --file main.bicep
.\deploy_bicep.ps1 -Profile beginner
```

Clean-run command sequence - post-Bicep data flow

```
cd ..\..\n\naz extension add --name datafactory --upgrade --yes\n\n$deployment = (Get-Content .\\infra\\bicep\\last-deployment.txt).Trim()\n.\\infra\\bicep\\run_after_bicep.ps1 -DeploymentName $deployment
```

Expected progression

ADLS upload -> ADF Bronze copy -> Event Hubs -> ADLS Bronze events -> Document Intelligence -> Search preparation.

Clean-run command sequence - deploy final models

```
cd infra\bicep
```

```
$env:POC08_CHAT_DEPLOYMENT="Phi-4-mini-instruct"
```

```
$env:POC08_CHAT_MODEL="Phi-4-mini-instruct"
```

```
$env:POC08_CHAT_MODEL_VERSION="1"
```

```
$env:POC08_CHAT_MODEL_FORMAT="Microsoft"
```

```
$env:POC08_CHAT_SKU="GlobalStandard"
```

```
.\deploy_bicep.ps1 -Profile models
```

Clean-run command sequence - final Search/RAG verification

```
cd ..\..\..
```

```
poetry run python -m scripts.verify_config --profile search
```

```
poetry run python -m scripts.verify_config --profile embeddings
```

```
poetry run python -m scripts.verify_config --profile foundry
```

```
poetry run python -m ai.rag.prepare_documents
```

```
poetry run python -m ai.rag.create_index
```

```
poetry run python -m ai.rag.index_documents
```

```
poetry run python -m ai.rag.query_search "When is a shipment considered delayed?"
```

```
poetry run python -m ai.rag.rag_answer "When is a shipment considered delayed?"
```

Clean-run command sequence - assistant and operations

```
poetry run python -m ai.agent.assistant --mode policy "What is the return window?"  
poetry run python -m ai.agent.assistant --mode metric "What is total revenue?"  
poetry run python -m ai.agent.assistant --mode mixed "What is total revenue and what is the delay  
policy?"  
  
poetry run python -m ingestion.events.send_malformed_event  
poetry run python -m monitoring.health_check
```


Final verification checklist - data platform

- Raw orders file exists in ADLS.
- ADF raw-to-bronze pipeline run is Succeeded.
- Bronze/Silver/Gold transformations are verified.
- Event Hubs received the sample shipment events.
- A JSONL event file exists under ADLS Bronze.
- One serving layer - Azure SQL, Synapse, or Fabric - returns the latest curated Gold data.
- Document Intelligence extraction result JSON exists.

Final verification checklist - AI platform

- Azure AI Search index `capstone-knowledge` exists.
- Policy documents and current Gold metrics are indexed.
- Hybrid/vector retrieval returns the expected source.
- Phi-4-mini-instruct and text-embedding-3-small are both deployed.
- RAG answer cites the correct retrieved source.
- Policy, metric, and mixed assistant tests pass.
- Unsupported question returns insufficient/unsupported context rather than invented facts.

Final verification checklist - engineering controls

- RBAC roles are documented and least-privilege intent is clear.
- Dataset owner/classification/lineage/access are documented.
- Monitoring evidence is captured.
- One controlled failure + recovery is captured.
- Azure DevOps YAML validates Python and Bicep and keeps deployment opt-in.
- Secrets are not committed to Git.
- Cost cleanup is planned before finishing the lab.

Cost cleanup

```
# From project root
.\infra\bicep\destroy_bicep.ps1 `
  -ResourceGroupName rg-poc08-capstone

# Type DELETE when prompted

# Verify deletion
az group exists --name rg-poc08-capstone
```

- Expected final result after deletion completes: `false`.
- Also terminate Databricks compute, ML endpoints/compute, paid Search, model deployments, Event Hubs, SQL, Synapse, and other reused billable resources if they are outside the dedicated POC resource group.
- Check Azure Cost Management after cleanup.

Five-minute final demo

1. Show the architecture and explain the batch, stream, document, serving, Search, and Foundry paths.
2. Show the synthetic raw input.
3. Show the successful ADF or pipeline run.
4. Show the curated Gold result.
5. Run Search retrieval and show the correct source.
6. Run the grounded assistant answer and show citation/tool source.
7. Show monitoring evidence.
8. Explain RBAC, read-only agent tools, and secret handling.
9. Show the cleanup command and cost-control plan.

Interview-ready project story

I built a small Azure Data + AI platform using synthetic retail data. ADF and Event Hubs handled batch and streaming ingestion into ADLS. Databricks created Bronze, Silver, and Gold data. Azure SQL, Synapse, or Fabric provided analytical serving. Document Intelligence handled unstructured invoices. Azure AI Search and Microsoft Foundry provided grounded RAG and a constrained read-only assistant. I applied Entra ID, RBAC, Key Vault patterns, monitoring, Bicep/CI-CD, controlled failure recovery, and cost cleanup across the stack.

Beginner completion summary

This is the point at which another beginner should be able to repeat the capstone from a clean project.

- **Local proof:** Poetry, tests, synthetic inputs, Bronze/Silver/Gold smoke test.
- **Infrastructure proof:** Bicep beginner deployment and resource verification.
- **Data proof:** ADLS, ADF, Event Hubs, Medallion, and one serving layer.
- **AI proof:** Document Intelligence, Search embeddings, Phi-based grounded RAG, constrained assistant.
- **Engineering proof:** governance, RBAC, monitoring, failure drill, CI/CD, cleanup.
- **Troubleshooting proof:** real deployment/runtime failures were diagnosed and converted into a repeatable runbook.